

Practical Generative AI for Teaching, Research & Productivity

Faculty Development Programme (FDP) · Powered by Akshara

6 Modules — POC Summary

Theory (PDF) · UI demos · Code structure · Live AI integration

Module	Folder	Port	Live AI
1 Intro GenAI	module-01-intro-genai	5171	UI lab
2 Prompt Engineering	module-02-prompt-engineering	5172	Yes
3 Responsible AI	module-03-responsible-ai	5173	UI lab
4 Agentic AI	module-04-agentic-ai	5174	Yes
5 AI Tools	module-05-ai-tools	5175	UI lab
6 Enterprise AINexLayer	module-06-enterprise-ainexlayer	5176	Yes (RAG)

Shared architecture

- Each module = separate Vite/React/TS/Tailwind app folder (as requested).
- Shared visual language: teal (#0F4C5C) + sand, Instrument Serif + DM Sans.
- ai-server/ — Express proxy so API keys stay server-side (never in browser).
- Providers: openai | azure (incl. Azure AI Foundry) | bedrock | gemini | vertex.
- Root .env / .env.example — configure AI_PROVIDER and keys; .env is gitignored.
- Vite proxy: /api, /health, /providers → http://localhost:8787
- Lead direction: UI demos + real AI APIs (wired strongest in Modules 2, 4, 6).

How to run

AI server: cd ai-server → npm run dev (port 8787)
Any module: cd module-0X-... → npm run dev (ports 5171–5176)

Module 1 — Introduction to Generative AI

Folder: *module-01-intro-genai* | **Port:** 5171 | **Live AI:** *Educational UI (API client ready; live AI not primary)*

POC use case: Live FDP walkthrough instead of static slides for Intro to GenAI.

Learning outcome: Participants understand how modern AI works and where it is applied.

Theory (from FDP PDF)

- Evolution of AI (symbolic → ML → deep learning → transformers → GenAI)
- Machine Learning vs Deep Learning
- What is Generative AI? Large Language Models (LLMs)
- Transformers (simplified attention concept)
- ChatGPT overview
- AI landscape: OpenAI, Gemini, Claude, Perplexity, Grok

UI — what we built

- FDP GenAI Lab shell (brand header + hero + Start exploring)
- Clickable AI evolution timeline
- Side-by-side ML vs Deep Learning comparison cards
- Token-by-token LLM prediction demo
- 3-step Transformers explainer
- AI landscape tool cards with faculty use cases

Code — what we implemented

- Vite + React 18 + TypeScript + Tailwind CSS
- src/App.tsx — interactive Module 1 content & state
- src/components/Shell.tsx — shared layout/branding
- src/lib/ai.ts — shared chat client (optional)
- vite.config.ts — port 5171 + proxy to ai-server

Module 2 — Mastering Prompt Engineering

Folder: *module-02-prompt-engineering* | **Port:** 5172 | **Live AI:** *YES — Run with real AI (Azure / OpenAI / Bedrock / Gemini / Vertex)*

POC use case: Workshop lab: improve prompts and call real Azure Foundry model (gpt-4.1).

Learning outcome: Faculty practice writing strong academic prompts and see live model output.

Theory (from FDP PDF)

- What is Prompt Engineering; characteristics of a good prompt
- Prompt frameworks; structured outputs; constraints
- Role prompting; Chain of Thought
- Zero-shot, one-shot, few-shot prompting
- Prompt chaining; prompt libraries
- Hands-on: lesson plans, question papers, assignments, research, coding, reports

UI — what we built

- Provider picker (Azure OpenAI, OpenAI, Bedrock, Gemini, Vertex)
- Technique selector (zero/one/few-shot, CoT, role, chaining)
- 6 academic lab tabs matching PDF hands-on list
- Weak prompt vs strong composed prompt comparison
- Run with real AI button + live output / error panel

Code — what we implemented

- src/App.tsx — prompt composer + chat() call
- src/components/ProviderPicker.tsx — health check + provider select

- src/lib/ai.ts — POST /api/chat via Vite proxy
- Composes ROLE + TASK + FORMAT + CONSTRAINTS + TECHNIQUE

Module 3 — Responsible AI & AI Limitations

Folder: *module-03-responsible-ai* | **Port:** 5173 | **Live AI:** *Educational UI (integrity scenarios; no live generation required)*

POC use case: Interactive ethics workshop: decide integrity cases and score authenticity process.

Learning outcome: Faculty can spot risks before trusting or submitting AI-assisted work.

Theory (from FDP PDF)

- AI hallucinations
- Bias in AI
- Detecting AI-generated content
- Making AI output authentic; fact verification
- Ethical AI usage; academic integrity

UI — what we built

- Hallucination example with verification tips
- Bias example cards + reflection prompt
- Authenticity checklist with live score
- Academic integrity scenario quiz (risk vs acceptable)
- Ethical usage closing banner

Code — what we implemented

- src/App.tsx — checklist state, scenario verdict logic
- Client-side only; no model call needed for core flow
- Shell layout + Module 3 branding

Module 4 — Agentic AI + Design Patterns

Folder: *module-04-agentic-ai* | **Port:** 5174 | **Live AI:** *YES — Run live AI agent + animated ReAct pattern replay*

POC use case: Demo ReAct conceptually, then generate a real lecture pack via Azure AI.

Learning outcome: Faculty see how agents plan, call tools, reflect, and collaborate.

Theory (from FDP PDF)

- Agentic AI fundamentals; agent architecture
- Tool/function calling; ReAct; planning & reasoning
- Reflection / self-correction; memory; RAG agents
- Multi-agent; supervisor/worker; human-in-the-loop
- MCP; workflows; state; evaluation; guardrails; production
- Agentic AI design patterns

UI — what we built

- Agent architecture building blocks
- Design patterns gallery (ReAct, RAG, supervisor, HITL, MCP, guardrails)
- Animated Thought → Action → Observation trace
- Run live AI agent for lecture-pack generation
- State / evaluation / production notes

Code — what we implemented

- src/App.tsx — pattern selector + timed ReAct animation
- handleLiveAgent() → chat() with agent system prompt
- ProviderPicker + ai.ts for real provider calls

Module 5 — AI Tools Demonstration

Folder: *module-05-ai-tools* | **Port:** 5175 | **Live AI:** *Educational UI (tool gallery + sample academic outputs)*

POC use case: Session guide for live tool demos: pick task, open real tool, compare with POC sample.

Learning outcome: Faculty know which tool fits which teaching/research task.

Theory (from FDP PDF)

- Live demos of ChatGPT, Gemini, Claude, Perplexity AI
- NotebookLM, Gamma (presentations), Napkin AI (diagrams), Canva AI
- Use real academic examples throughout

UI — what we built

- Pick a teaching task → recommended tools
- Tools gallery with category, strengths, sample prompt
- Simulated demo output panel per tool
- Comparison matrix table

Code — what we implemented

- src/App.tsx — TOOLS + TASKS data model and filtering
- Task→tool recommendation logic (client-side)
- Responsive gallery + comparison table

Module 6 — Enterprise AI with AINexLayer

Folder: *module-06-enterprise-ainexlayer* | **Port:** 5176 | **Live AI:** *YES — Live RAG Q&A; over seeded institutional docs + multi-provider*

POC use case: Enterprise POC: ask policy questions grounded in campus docs with role checks.

Learning outcome: Show secure institutional AI (RAG, RBAC, workflows) vs public chatbots.

Theory (from FDP PDF)

- From public AI tools to Enterprise AI
- Enterprise assistant; document intelligence; multi-LLM
- Enterprise search; knowledge base; RAG; AI agents
- Workflow automation; RBAC; on-prem & cloud
- Use cases: faculty/research assistants, policy Q&A, student support, admin automation
- AI Lab + AINexLayer for secure institution-wide adoption

UI — what we built

- Public → Enterprise → AI Lab journey cards
- Role switcher (faculty / admin / student) with permissions
- Knowledge base browser + document intelligence panel
- RAG query box with retrieved sources + live answer
- Workflow automation stepper
- Institutional use cases + industry cards

Code — what we implemented

- src/App.tsx — seeded DOCS[], keyword retrieval, RBAC gate
- runRag() builds context → chat() with grounded system prompt
- ProviderPicker for Azure/OpenAI/etc.
- Simulated institutional policies/handbook/SOPs as KB

Summary for stakeholders

- Deliverable matches lead ask: interactive UI demos per module + real AI APIs.

- Theory content is aligned to Faculty Development Programme (FDP).pdf.
- Modules 1, 3, 5 emphasize teaching/workshop UX; 2, 4, 6 call live models.
- Azure AI Foundry configured via AZURE_OPENAI_* (services.ai.azure.com + deployment).
- Next optional upgrades: wire live AI into Modules 1/3/5, deeper tool-calling agents, real vector RAG.